

Linux: Backup

D. Leeuw

10 juni 2026

© 2026 Dennis Leeuw



Dit werk is uitgegeven onder de Creative Commons BY-NC-SA Licentie en laat anderen toe het werk te kopiëren, distribueren, vertonen, op te voeren, en om afgeleid materiaal te maken, zolang de auteurs en uitgever worden vermeld als maker van het werk, het werk niet commercieel gebruikt wordt en afgeleide werken onder identieke voorwaarden worden verspreid.

1 Over dit Document

1.1 Leerdoelen

Dit document leert de lezer:

- hoe je bestanden en directories veilig kan stellen
- hoe je hele disks veilig kan stellen

1.2 Voorkennis

Van de lezer wordt verwacht dat deze kennis heeft van:

- de werking van linux commando's
- opslag systemen zoals harddisks, ssd en USB-sticks en hoe deze op Linux worden aangesproken

2 Introductie

Digitale data is kwetsbaar. Je kan het per ongeluk overschrijven of weggooien. Het kan gelockt worden door een crypto-locker of gestolen worden door een kwaadwillende. Data kan ook verloren gaan door diefstal, brand of het defect gaan van opslagsystemen. Het terug krijgen van de juiste data kan een heleboel werk opleveren als we geen goede backups hebben van de data of zelfs onmogelijk blijken te zijn.

Moderne systemen zoals smartphones zijn vaak verbonden met cloud-opslagsystemen waarbij er een automatische backup gemaakt wordt van de belangrijkste data. Dat geldt voor Linux systemen vaak niet. Als we onze data veilig willen stellen dan zullen we dat zelf moeten doen en moeten we erover nadenken hoe we dat gaan doen. Dit document bevat een aantal commando's die je kunnen helpen bij het veiligstellen van jou data of de data van gebruikers waarvoor je verantwoordelijk bent.

3 Het maken van een kopie

Met `cp` kan je een kopie (copy) maken een bestand. Op deze manier kan je zorgen voor:

verschillende versies Maak een kopie van een bestand en werk daarna pas verder in een document. Je hebt dan een oudere versie om naar terug te gaan.

een veiligheidskopie Kopieer een bestand of bestanden naar een USB-disk en verwijder de USB-disk. Op deze manier heb je een off-site backup.

4 GNU-ZIP

Bestanden kunnen behoorlijk wat ruimte innemen op je disk. Een manier om te zorgen dat ze kleiner worden is door gebruik te maken van compressie-software. Eén van de meest gebruikte tools hiervoor op een GNU/Linux systeem is GNU-ZIP ofwel het `gzip` commando.

5 Tape archive

Linux kent één commando dat bij elke beheerder bekend is voor het maken van backups. Deze tool is `tar`. De naam is een afkorting van Tape Archive, wat gelijk de herkomst verraad. Backups werden en worden nog vaak op tape gemaakt en dat is waar `tar` voor geschreven was. Maar omdat alles een bestand is kan tar inplaats van naar tape ook naar een bestand schrijven.

Naast de naam van de tool is tar ook de naam van het bestandsformaat van `tar`. Data wordt op tape weggeschreven als één lange rij enen en nullen. Als je verschillende bestanden achter elkaar zet moet er ergens een index zijn die zegt waar een bestand begint en waar het eindigt. De manier waarop dit beschreven wordt in een tar-archive heet het tar-bestandsformaat en een archief met bestanden wordt vaak een tarball genoemd.

We beginnen met het maken van een archief als bestand:

```
tar cvf my_archive.tar <directory_met_bestanden>
```

Met `c` geven we aan dat we een archief willen maken (create). De `v` staat voor verbose, zodat we kunnen zien wat `tar` toevoegd aan het archief en `f` vertelt waar we data naartoe willen schrijven, in dit geval naar `my_archive.tar`. Met de `.tar` extensie vertellen we dat een archief is in het tar-bestandsformaat. Als laatste geven we op wat we in het archief willen stoppen, we kiezen voor een directory waarin wat bestanden staan.

Met `x` (extract) kunnen we bestanden uit het archief halen. Dat kan één enkel bestand zijn of we kunnen het hele archief uitpakken.

```
mkdir extract
cd extract
tar xvf ../my_archive.tar
```

We hebben een nieuwe directory aangemaakt en het hele archief uitgepakt in deze nieuwe directory. Feitelijk hebben we nu een kopie van onze data.

De oorspronkelijke data neemt ruimte in op de disk, het archief neemt de ruimte in van de data plus wat administratieve overhead en de uitgepakte data neemt weer de ruimte in van de data. Het wordt tijd dat we gaan opruimen. Begin met het weggooien van de `extract-directory`.

Gebruik `ls` om te achterhalen hoe groot `my_archive.tar` is. We kunnen `gzip` gebruiken om het archief te comprimeren:

```
gzip my_archive.tar
```

Bekijk weer de grootte van het archief bestand. Het zal nu een stukje kleiner zijn. De `tar` tool kan dit ook in 1x doen:

```
tar zcvf my_archive.tar.gz <directory_met_bestanden>
```

De `z` geeft aan dat we GNU-ZIP (`gzip`) gebruiken. Je kan ook de `j` gebruiken voor compressie met `bzip2` of de `J` voor compressie met `xz`. Denk erom dat als je een andere compressie gebruikt dat je dan ook de extensie van het archief aanpast.

Bij het uitpakken van een gecomprimeerd archief kan je dezelfde letters gebruiken om te zorgen dat `tar` eerst de tarball uncompressed en daarna uitpakt.

6 Carbon Copy

Met `dd` kan bestanden, delen van bestanden, disk of delen van disks kopieëren. Je geeft `dd` een bestand om van te lezen en een bestand waar naar geschreven moet worden. Omdat alles een bestand is kan het een bestand op disk zijn, maar ook bijvoorbeeld een harddisk. Stel we hebben een USB-stick in onze computer zitten van 8GB en deze wordt door Linux `/dev/sdb` genoemd. Dan kunnen we met `dd` een kopie maken van deze disk:

```
dd if=/dev/sdb of=~/.sdb.img
```

De optie `if` heeft als input-bestand de disk `/dev/sdb`, de optie `of` heeft als output-bestand een document met de naam `sdb.img`. `dd` zal aan het begin van `/dev/sdb` beginnen met lezen en bit voor bit de data in `sdb.img` wegschrijven. Als `dd` aan het einde van `sdb` is is hij klaar.

Stoppen we hierna een lege USB-stick, ook van 8GB, op de plaats van de stick die we net gelezen hebben dan kunnen we het omgekeerde doen:

```
dd if=~/.sdb.img of=/dev/sdb
```

We schrijven nu de image terug naar een andere USB-stick. Effectief hebben we nu een kopie gemaakt van onze USB-stick.

Aan het begin van deze cursus heb je een ISO-bestand van de Debian site gedownload (debian.iso). Dit heet een ISO-image en ISO is een afkorting voor het bestandsformaat dat op CD-ROMs gebruikt wordt (ISO-9660). Deze image heb je gebruikt om je Debian systeem te installeren. Je had hem ook met dd op een USB-stick kunnen zetten en dan had je vanaf die USB-stick kunnen booten.

7 Off-site backup

Een universele tool om data te kopiëren is **rsync**. Met **rsync** kan je lokaal bestanden kopiëren, maar je kan bestanden ook naar een andere server kopiëren. Naast het maken van een kopie kan rsync bestanden ook in sync houden. Dus alleen een kopie maken als een bestand gewijzigd is. Hiermee is **rsync** de tool om data veilig te stellen.

De **rsync** tool gebruikt de standaard SRC DST syntax zoals meerder Linux tools. Maar de SRC of DST kan ook een host of user@host zijn zodat je data van of naar een remote system kan kopiëren.

Een voorbeeld zou kunnen zijn:

```
rsync -r Documents user@127.0.0.1:~/Documents.copy
```

De **-r** is voor recursive zodat we de complete directory kopiëren.

Als we data over willen zetten waarbij alle rechten en alle time-stamps behouden blijven dan kunnen we de archive mode gebruiken. Ideaal voor backups.

```
rsync -a Documents user@127.0.0.1:~/Documents.copy
```

De **rsync** tool is een heel voorzichtige tool. Dat betekent dat bestanden niet zomaar weggegooid worden. Updates wil hij wel uitvoeren, maar voor het verwijderen van bestanden die aan de bron verwijderd zijn moet je **rsync** specifiek de opdracht daartoe geven. Voor backups natuurlijk perfect, dan blijven oude bestanden bestaan en kan je ze later terug zetten. Maar als je twee websites in sync wil houden, dan wil je dat oude bestanden verwijderd worden. Zie de man-page voor alle mogelijk opties, wij gebruiken er hier maar één.

```
rsync -a --delete-after Documents user@127.0.0.1:~/Documents.copy
```

De extra optie zegt tegen **rsync** dat hij eerst alle data in sync moet brengen en daarna pas verwijderde bestanden mag weggooien. Dit voorkomt bijvoorbeeld dat code of CSS-bestanden die van elkaar afhankelijk zijn weggegooid worden voordat een vervanger geplaatst is.

De data die wordt verstuurd gaat as-is over de lijn. Voor de veiligheid zouden we data encrypt over de lijn willen laten gaan. Daarvoor kunnen we een optie aan **rsync** meegeven die inplaats van de standaard shell **ssh** gebruikt.

```
rsync -a --rsh="ssh -i /home/dennisnc/.ssh/hostname-key -l dennis"  
Documents user@127.0.0.1:~/Documents.copy
```

We vertellen hier tegen rsync dat de remote shell **ssh** is en dat onze lokale gebruiker *dennisnc* een private key heeft voor de server waarop ingelogd moet worden met de remote gebruiker *dennis*. Op deze manier hebben we een veilige overdracht van data.

Index

backup, 3
 tar, 3

cp, 2

dd, 4

rsync, 5

tar, 3
 bestandsformaat, 3
tarball, 3

versioning, 2